Release, distribution, and push notifications

Build modes, signing, the stores, CI/CD, and FCM

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Autumn 2026

TODAY

What you should leave with

Produce a real build

Debug, profile and release, flavors, icons, and a version number the stores accept.

Sign it correctly

Android keystores and Play App Signing, iOS certificates and provisioning profiles.

Deliver it

Play Console and App Store Connect, internal testing and TestFlight, GitHub Actions and fastlane.

Push to a device

FCM tokens, the three app states, and what a notification payload may carry.

PART 1 · BUILDING

Making an installable build

Build modes, flavors, icons, and version numbers

Three build modes, three purposes

Debug

JIT compiled, assertions on, hot reload, service extensions, a debug banner. Large and slow on purpose. Never measure performance here.

Profile

AOT compiled with tracing kept, so DevTools can still see frames and CPU. Runs on a real device only, never on an emulator.

Release

AOT compiled, assertions stripped, no debugging hooks, tree shaken icons. This is the only mode a user ever runs.

The mode is a compile-time decision. It changes the Dart compiler, the asserts, and the size of the binary.

A bug that only appears in release is usually an assert or a debug-only code path. Reproduce with `flutter run --release` before you blame the store build.

The artifacts each store wants

```
# Android: the format Play expects
flutter build appbundle --release
# Sideloading and CI smoke tests only
flutter build apk --split-per-abi
# iOS: an archive, then upload it
flutter build ipa --release
```

An app bundle is not an installable app.
You upload one `.aab`, and Play generates the APK for each device: its ABI, screen density and language only. `flutter build ipa` writes an Xcode archive plus an `.ipa`. You upload it with Transporter or from Xcode Organizer.

Ship the bundle, keep the debug symbols. Archive the mapping and symbol files from every release build, or your crash reports stay unreadable.

Flavors: one codebase, several apps

```
flutter run --flavor dev \
  -t lib/main_dev.dart

flutter build appbundle --flavor prod \
  -t lib/main_prod.dart \
  --dart-define=API_URL=https://api.dev
```

Each flavor gets its own application id, its own icon and its own name, so development and production can sit side by side on the same phone.

Where it is configured

Android: product flavors in the Gradle build file. iOS: one scheme and one build configuration per flavor, in Xcode.

Read the values back with `String.fromEnvironment`. A `--dart-define` is baked in at compile time, so it is not a secret.

Icons, names, and the splash screen

```
flutter_launcher_icons:
  image_path: "assets/icon.png"
  android: true
  ios: true
  adaptive_icon_background: "#101010"
  adaptive_icon_foreground: "fg.png"
```

Generate, do not hand-place. One square source image, one command, and every density and every iOS size is written for you. Adaptive icons need a foreground and a background layer, because the launcher masks them into its own shape. `flutter_native_splash` writes the native launch screen. Flutter cannot paint before the engine starts.

The app icon may not have a transparent background. iOS rejects the upload for it. Export the source image on a solid color.

Version name and build number

```
# pubspec.yaml
version: 1.4.0+27
```

```
flutter build appbundle \
  --build-name=1.4.0
    --build-number=28
```

PART	ANDROID	IOS
1.4.0	versionName	CFBundleShortVersionString
27	versionCode	CFBundleVersion

The name is for humans and may repeat. The number is for the store, must be an integer, and must increase with every upload.

A build number is spent the moment you upload it. Both stores refuse a second upload with a number they already have, even if you deleted the first one. Let CI set it from the run number.

PART 2 · SIGNING

Proving the update is yours

Keystores, Play App Signing, certificates and profiles

The Android keystore

```
keytool -genkey -v -keystore upload.jks \
  -keyalg RSA -keysize 2048 -validity 10000
    \
  -alias upload
```

```
# android/key.properties  (never committed)
storeFile=/Users/you/keys/upload.jks
storePassword=...
keyAlias=upload
keyPassword=...
```

Every Android app is signed.
The signature is the identity. Android installs an update only when it is signed by the same key as the installed app.
Gradle reads `key.properties` and applies the release signing config. Without it, `flutter build` silently falls back to the debug key.

Play App Signing

You hold the upload key

You sign the bundle with it. Play verifies the signature, then strips it. It only proves the upload came from you.

Google holds the app signing key

Play re-signs the generated APKs with it. That key is what devices check, and it never leaves Google.

The upload key is replaceable

Lose it and you register a new one through support. Lose a self-managed app signing key and the app can never be updated.

Play App Signing is required for new apps, and it is what makes per-device APK splitting possible.

Back up the keystore and its passwords the day you create them. A password manager entry and an encrypted copy off your laptop. Not the repository.

SIGNING

iOS: four things that must agree

PIECE	WHAT IT IS	WHERE IT LIVES
Certificate	Apple's signature on your public key	Keychain, exported as `.p12`
App ID	The bundle identifier and its capabilities	Developer portal
Devices	UDIDs allowed to run a test build	Developer portal
Profile	Certificate, App ID and devices, in one file	`.mobileprovision`

Xcode manages all four for one developer. A team shares them deliberately: `fastlane match` keeps them in a private encrypted repository.

Certificates expire. A build that worked last year fails today for no other reason.

What never goes into the repository

⚠ Keep these out of Git

Keystores and `key.properties`. Signing certificates and `.p12` exports. Provisioning profiles. App Store Connect API keys. Play service account JSON. Any `.env` with a real secret.

A secret in Git is public.
Deleting the file does not help: the object stays in history, and on a public repository it is scraped within minutes.
If it happens: rotate the credential first, then clean the history. In that order. Rotation is what actually stops the damage. Store the real values in the CI provider's encrypted secrets and in a password manager for humans.

Write the ignore rules before you create the keys. That is the only moment when the repository is still guaranteed clean.

PART 3 · STORES

Getting past review

Play Console, App Store Connect, test tracks, and privacy

STORES

Google Play Console: the first release

1. Create the app and fix the package name. It is permanent.
2. Complete the store listing. Descriptions, icon, feature graphic, screenshots.
3. Answer the questionnaires. Content rating, target audience, ads, Data safety.
4. Upload the signed bundle to a track, with release notes.
5. Roll out in stages, and watch the crash rate as it climbs.

Review takes days, not minutes, and a new developer account waits longest.

App Store Connect: the first release

1. Register the App ID and create the app record. Bundle identifier and SKU, both permanent.
2. Upload a build. From Xcode Organizer or Transporter. It appears after processing.
3. Fill in the metadata. Screenshots per device size, description, support and privacy URLs.
4. Answer App Privacy and export compliance, attach the build, submit.
5. Release manually or in phases once it is approved.

Give the reviewer a demo account and notes for anything behind a login. A reviewer who cannot get in rejects the app.

STORES

Test tracks before anyone real sees it

TRACK	STORE	WHO GETS IT	REVIEW
Internal testing	Play	a small list of accounts you name	minimal, available quickly
Closed testing	Play	testers on a list or in a group	yes
Open testing	Play	anyone with the opt-in link	yes
TestFlight internal	App Store	members of your team	none
TestFlight external	App Store	invited testers or a public link	Beta App Review

Every release goes through a test track first. A build that installs and launches on ten real devices has already caught the failures an emulator hides.

STORES

Why apps get rejected

It crashed for the reviewer

The most common reason. Test the exact build you submitted.

The reviewer could not sign in

No demo account, a dead link, or a phone code they cannot receive.

Placeholder content

Lorem ipsum, dead links, or a support URL that does not resolve.

Permissions without a reason

A permission you request but never use.

Misleading metadata

Screenshots of features that do not exist.

Too little to be an app

A thin wrapper around a website.

Privacy declarations and permissions

You declare data before you collect it. Play calls it the Data safety form, Apple calls them privacy labels. Both ask what you collect, why, and whether you share it. The declaration must match the app, including whatever your analytics and crash SDKs send. An SDK that collects an advertising identifier is your collection, not theirs.

On the device

iOS needs a purpose string in `Info.plist` for every sensitive API, shown verbatim in the prompt. Android declares permissions in the manifest and asks at runtime.

Ask at the moment the permission is needed, with the reason on screen first. A cold prompt at launch is denied, and a denial is hard to reverse.

PART 4 · AUTOMATION

Nobody releases from a laptop

GitHub Actions, fastlane, Codemagic, and code push

What a release pipeline does

On every pull request

Format check, `flutter analyze`, unit and widget tests. Merging is blocked while any of them is red.

On every merge to main

Build the release artifacts for both platforms so a broken build is found in minutes, not at release time.

On a version tag

Sign, set the build number, upload to internal testing and TestFlight, and post the link to the team channel.

The pipeline is also the honest answer to ”it works on my machine”. It builds from a clean checkout every time, with no local caches and no local keys.

Automate the boring half first. Tests and analysis on pull requests pay for themselves in a week. Automated store upload can wait until you release often enough to care.

AUTOMATION

A GitHub Actions workflow

```yaml
name: ci
on: [pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: subosito/flutter-action@v2
      - run: flutter pub get
      - run: dart format --set-exit-if-changed .
      - run: flutter analyze
      - run: flutter test
```

Format, analyze and test are what you run locally: a red pipeline is never a surprise.

Secrets in the pipeline

```
- name: Restore the keystore
  env:
    KEYSTORE: ${{ secrets.KEYSTORE_B64 }}
  run: |
    echo "$KEYSTORE" | base64 -d \
      > android/upload.jks
```

Binary secrets travel as base64. Store them in the provider's encrypted secrets, decode them into the workspace, and let the job delete the workspace afterwards.
Never echo a secret, never print it in a failing step, and never expose secrets to a workflow triggered by a fork's pull request.

A secret printed once in a build log is a burned secret. Rotate it. Deleting the log does not help.

AUTOMATION

fastlane and Codemagic

	FASTLANE	CODEMAGIC
What it is	Ruby lanes you run anywhere	Hosted CI built around Flutter
Signing	`match` shares certificates through a private repo	Managed in the UI, on a machine with Xcode
Upload	`supply` for Play, `pilot` and `deliver` for Apple	Publishing to both stores is a built-in step
Fits	teams that want the pipeline in their repo	teams that do not want to run macOS runners

They are not exclusive: a common setup is GitHub Actions for tests and fastlane for everything that talks to a store.

AUTOMATION

Code push, in one slide

Shorebird patches Dart code on installed apps.
You publish a patch, the app downloads it, and the fix is live on the next launch without a store review.
It covers Dart code and assets. Anything native, a new plugin, a new permission, or a version bump still needs a real store release.

⚠ Use it narrowly

A patch must stay inside what the stores allow: a fix to the app you shipped, not a new product. Treat it as a hotfix channel, and keep every patch in Git.

It also splits your users across two code paths. Know which patch a crash report came from before you debug it.

PART 5 · PUSH

Waking an app that is closed

FCM, APNs, tokens, and the three app states

How a push actually arrives

Your backend decides

It calls the FCM HTTP v1 API with a service account credential. The app never sends a push, because the app cannot hold that key.

FCM routes it

Firebase Cloud Messaging picks the transport: Google Play services on Android, APNs on iOS. One API, two networks.

The device delivers it

The system wakes your app or draws the notification itself, depending on the payload and on what the app is doing.

Delivery is best effort. Messages can be delayed by battery saving, collapsed with a newer one, or dropped while the device is offline.

A push is a hint, not a transport. Send an identifier and let the app fetch the real data over HTTP. Never treat a push as your only copy of a state change.

Setup and the registration token

```
final fm = FirebaseMessaging.instance;
await fm.requestPermission();

final token = await fm.getToken();
await api.registerDevice(token);

fm.onTokenRefresh.listen(api.register);
```

The token is the address.
One token per app install. It changes on reinstall, on a data clear, and sometimes on its own.
Store it against the user and the device, delete it on sign-out, and drop it when the send API reports it unregistered. iOS also needs the APNs auth key in the Firebase project, plus push and background modes enabled in Xcode.

Notification messages and data messages

```
message:
  token: "..."
  notification:
    title: "New comment"
    body: "Ana replied to your task"
  data:
    taskId: "8412"
```

A `notification` block is drawn by the operating system when your app is not in the foreground. Your code does not run until the user taps it. A `data` block is never drawn. It is handed to your app, so you decide what to show. On iOS it needs the background flag and is delivered at the system's convenience.

Sending both is the usual choice.

The system shows something reliable, and the data payload tells you which screen to open on tap.

Handling the three app states

```
@pragma('vm:entry-point')
Future<void> bg(RemoteMessage m) async {
  await Firebase.initializeApp();
}
FirebaseMessaging.onBackgroundMessage(bg);

fm.onMessage.listen(showLocal);
fm.onMessageOpenedApp.listen(openScreen);
final m = await fm.getInitialMessage();
```

Foreground: nothing is drawn for you. Show a local notification, or update the screen. Background: the system draws it, and `onMessageOpenedApp` fires on tap. Terminated: the app starts cold, so `getInitialMessage` is the only place the tapped message appears.

Background handler

Top level or static, annotated, and it initializes Firebase itself.

Asking for permission, and what to send

You get one prompt.
iOS has always required permission. Recent Android versions require it too. A denial is final until the user walks into system settings.
Ask after the user has done something that a notification would follow: created a task with a due date, joined a conversation. Explain the value on your own screen first. Group related messages, collapse duplicates, and respect quiet hours. An app that pushes daily noise gets its notifications switched off, then deleted.

⚠ Not in a payload

Passwords, tokens, one-time codes, health or financial details, anything private enough to matter on a lock screen. The payload passes through Google and Apple, and is stored on the device.

Send an identifier and a neutral title. Fetch the sensitive content after the user opens the app and is authenticated.

CHECKLIST

Before you ship

- [ ] Release build tested on a physical device, not only an emulator
- [ ] Build number increased, release notes written
- [ ] Signing keys backed up outside the repository
- [ ] Crash reporting and analytics live in the release build
- [ ] Privacy declarations match what the app actually sends
- [ ] Every permission has a purpose string and a reason on screen
- [ ] A demo account and review notes are attached
- [ ] A staged rollout, and someone watching the crash rate

Run this list from a written checklist, not from memory. The one you skip is the one that fails.

COURSE RECAP

Fourteen weeks in one table

WK	TOPIC
1	Orientation, Git, build strategies
2	Dart, null safety, toolchain
3	Widgets and layout
4	Async Dart, coding agents
5	Debugging, state part 1
6	BLoC and Riverpod
7	HTTP, JSON, code generation

WK	TOPIC
8	Firebase, flags, GraphQL
9	Observability, local storage
10	Auth, OAuth, App Check
11	Routing, permissions, sockets
12	AI in the app
13	UI polish and testing
14	Release, distribution, push

LAB TEST

The lab test and your grade

Lab 7 is the practical test.
Individual, 90 minutes, your own laptop, internet allowed. Agents are allowed, and you must be able to explain every line you submit.
A one-page spec of a small app with four requirements: a screen built from the layout widgets, state in a Cubit, one network call with a model class, one navigation with `go_router`. One point per requirement, partial credit for a requirement that is present but incomplete. Submit on branch `lab-7` before the end.

COMPONENT	POINTS
Lab assignments, from the repo	5
Practical lab test, Lab 7	4
Participation	1
Pass	5 of 10

Every requirement maps to a lab you already did. Nothing in the test is new material.

RECAP

Three things to remember

1. **Release is a process, not a build command.** Signing, versioning, forms and review take longer than the compile.
2. **Guard the keys, automate the rest.** A lost signing key is unrecoverable. Everything else can be scripted.
3. **A push is a hint.** Send an identifier, fetch the content, and never put a secret in a payload.

FURTHER READING

docs.flutter.dev/deployment/android · deployment/ios · Firebase Cloud Messaging · docs.fastlane.tools

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel